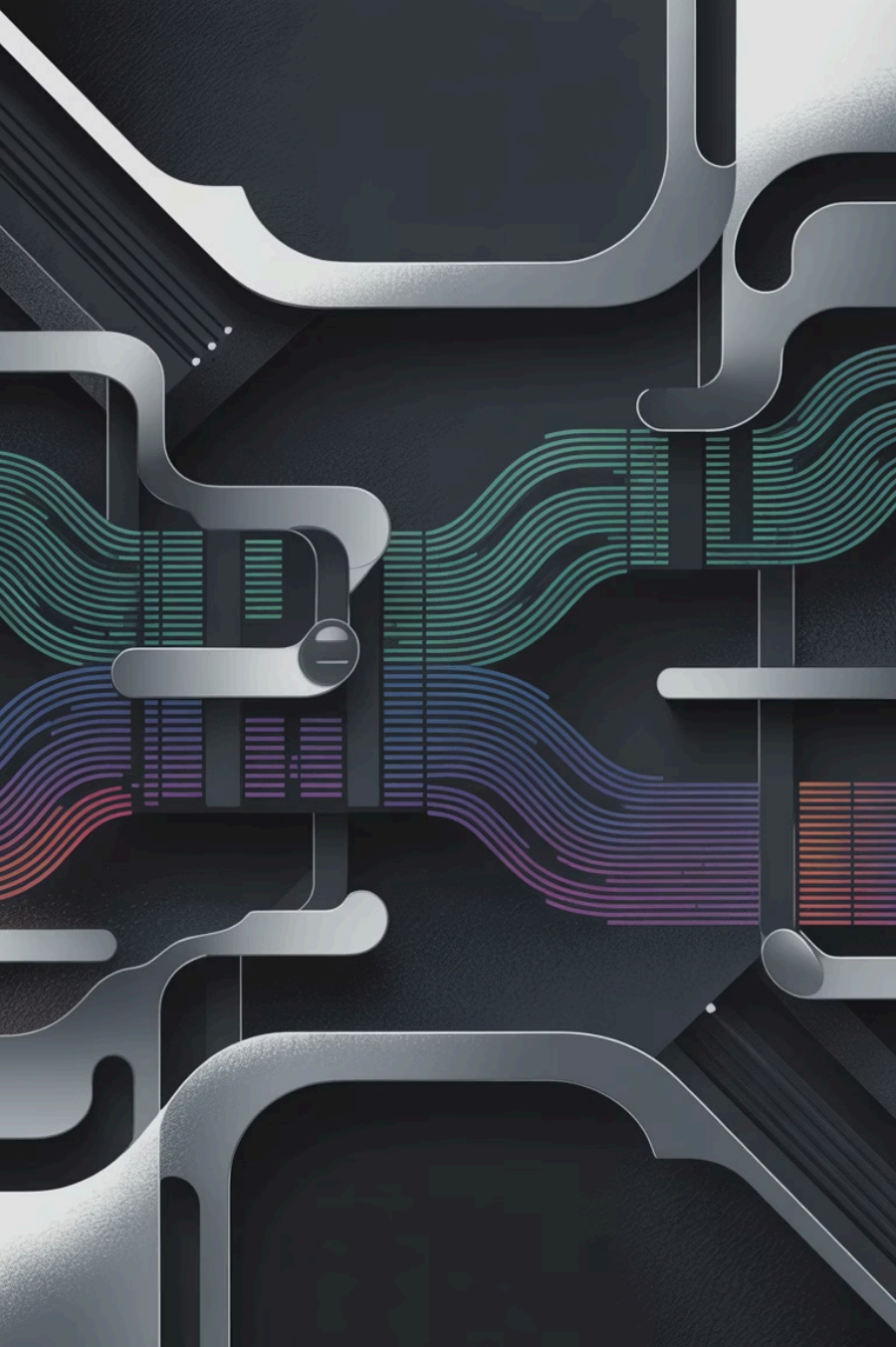




Module 05 - Data Engineering for LLM Pipelines

Building the Foundation for AI Engineering Excellence

Module Learning Journey

What We'll Cover

01

Boolean Indexing Fundamentals

Understanding conditional selection with single and multiple conditions

02

Advanced Query Methods

Leveraging `.query()` for readable, SQL-like data filtering

03

Chained Masks & Complex Logic

Building sophisticated filters for multi-condition scenarios

04

Location-Based Selection with `.loc`

Precise row and column selection using label-based indexing

05

Real-World Application

Hands-on lab filtering datasets for LLM preprocessing

Why This Matters

Data Quality Determines Model Quality

In LLM engineering, your model is only as good as your training data. Effective filtering and selection techniques allow you to isolate high-quality examples, remove noise, and create targeted datasets that improve model performance.

Whether you're building fine-tuning datasets, cleaning scraped web data, or validating API responses, mastering Pandas selection is non-negotiable. These techniques form the foundation of your data preprocessing pipeline.



Boolean Indexing: The Foundation

Boolean indexing is the most fundamental filtering technique in Pandas. It works by creating a Series of True/False values that correspond to each row in your DataFrame, then using that mask to select only the rows where the condition is True.

```
import pandas as pd

# Load your dataset
df = pd.read_csv('training_data.csv')

# Simple boolean mask
mask = df['rating'] > 4.0
filtered_df = df[mask]

# Shorthand (most common)
high_quality = df[df['rating'] > 4.0]
```

This technique is essential when filtering training examples by quality scores, removing outliers, or selecting specific categories from your dataset.

Combining Multiple Conditions

Real-world data filtering rarely involves just one condition. You'll often need to combine multiple criteria using logical operators. In Pandas, use `&` for AND, `|` for OR, and `~` for NOT.

```
# Multiple conditions with & (AND)
quality_data = df[(df['rating'] > 4.0) & (df['word_count'] > 100)]

# OR condition with |
flagged = df[(df['contains_profanity'] == True) | (df['spam_score'] > 0.8)]

# NOT condition with ~
clean_data = df[~df['is_duplicate']]

# Complex combination
ideal_examples = df[
    (df['rating'] > 4.5) &
    (df['word_count'].between(50, 500)) &
    ~(df['language'] == 'unknown')
]
```

Critical: Always use parentheses around each condition. Python's operator precedence requires them, and omitting parentheses will cause unexpected results.

Common Pitfall

Why Parentheses Matter

✗ Wrong (Will Fail)

```
# This will raise an error
df[df['age'] > 18 & df['active'] == True]

# Why? Operator precedence
# Python evaluates as:
# df['age'] > (18 & df['active']) == True
```

✓ Correct

```
# This works properly
df[(df['age'] > 18) & (df['active'] == True)]

# Python evaluates as:
# (df['age'] > 18) & (df['active'] == True)
```

📌 **Pro Tip:** When combining conditions, wrap each comparison in parentheses. This simple habit will save you hours of debugging.

The .query() Method: SQL-Like Syntax

The `.query()` method provides a more readable alternative to boolean indexing, especially for complex conditions. It accepts a string expression that looks similar to SQL WHERE clauses, making it intuitive for those with database experience.

```
# Traditional boolean indexing
result = df[(df['rating'] > 4.0) & (df['word_count'] > 100) & (df['language'] == 'en')]

# Same filter using .query()
result = df.query('rating > 4.0 and word_count > 100 and language == "en"')

# Variables can be referenced with @
min_rating = 4.0
min_words = 100
result = df.query('rating > @min_rating and word_count > @min_words')
```

Notice how `.query()` uses `and` and `or` instead of `&` and `|`, and doesn't require parentheses around each condition. This makes complex filters significantly more readable.

When to Use `.query()` vs Boolean Indexing

`.query()` Advantages

- More readable for complex multi-condition filters
- Shorter syntax reduces typing and potential errors
- Easier to construct filters dynamically from strings
- Natural for those familiar with SQL

Boolean Indexing Advantages

- Better for single conditions or simple filters
- Works with any Python expression or function
- Slightly faster performance on small datasets
- More explicit and familiar to Python developers

In practice, use whichever method makes your code most readable. Many engineers use boolean indexing for simple filters and `.query()` when combining three or more conditions.

Chained Masks: Building Complex Logic

Chained masks allow you to build complex filtering logic step-by-step, making your code more readable and easier to debug. This approach is particularly valuable when you need to apply different filters based on intermediate results or when building filters dynamically.

```
# Create individual masks
high_quality = df['rating'] > 4.5
sufficient_length = df['word_count'] > 100
english_only = df['language'] == 'en'
no_duplicates = ~df['is_duplicate']

# Combine masks step by step
initial_filter = high_quality & sufficient_length
language_filter = initial_filter & english_only
final_filter = language_filter & no_duplicates

# Apply the final mask
clean_dataset = df[final_filter]
```

This technique is especially useful when different team members own different quality criteria, or when you need to log how many rows each filter removes.

Practical Example

Building a Training Dataset Filter

```
def filter_training_data(df, config):
    """Filter dataset for LLM fine-tuning based on quality criteria"""

    # Step 1: Quality score threshold
    quality_mask = df['human_rating'] >= config['min_rating']
    print(f"After quality filter: {quality_mask.sum()} examples")

    # Step 2: Length requirements
    length_mask = quality_mask & df['text_length'].between(
        config['min_length'],
        config['max_length']
    )
    print(f"After length filter: {length_mask.sum()} examples")

    # Step 3: Remove problematic content
    safety_mask = length_mask & ~df['flagged_content']
    print(f"After safety filter: {safety_mask.sum()} examples")

    # Step 4: Ensure variety — keep only well-represented categories
    # .map() looks each row's category up in the value_counts and returns
    # the count ALIGNED TO df's index.
    category_counts = df['category'].map(df['category'].value_counts())
    final_mask = safety_mask & (category_counts >= 10)
    print(f"Final dataset: {final_mask.sum()} examples")

    return df[final_mask]
```



The .loc Accessor: Label-Based Selection

While boolean indexing selects entire rows, `.loc` gives you precise control over both rows and columns using label-based indexing. This is essential when you need to select specific fields from filtered data or update values in place.

```
# Select rows and columns
df.loc[df['rating'] > 4.0, ['text', 'rating', 'category']]

# Select single column from filtered rows
high_quality_text = df.loc[df['rating'] > 4.0, 'text']

# Select multiple columns using a list
features = df.loc[df['is_training'] == True, ['text', 'label', 'metadata']]

# Update values for filtered rows
df.loc[df['word_count'] < 10, 'quality_flag'] = 'too_short'
```

The power of `.loc` becomes clear when preparing data for machine learning: you can filter rows based on quality criteria while simultaneously selecting only the columns needed for training.

.loc vs .iloc: Understanding the Difference

.loc (Label-Based)

```
# Uses column names and index labels  
df.loc[0:5, 'text':'rating']
```

```
# Works with boolean masks  
df.loc[df['active'] == True, 'features']
```

```
# Inclusive of end point  
df.loc[0:5] # includes row 5
```

Use `.loc` when working with meaningful row indices or column names.
This is the most common accessor in production code.

.iloc (Position-Based)

```
# Uses integer positions  
df.iloc[0:5, 2:7]
```

```
# Works with numeric indices  
df.iloc[[0, 5, 10], [1, 3]]
```

```
# Exclusive of end point  
df.iloc[0:5] # excludes row 5
```

Use `.iloc` when position matters more than labels, such as splitting datasets or selecting the first N rows.

Common Selection Patterns for LLM Data

Train/Test Split by Date

```
train = df.loc[
    df['date'] < '2024-01-01',
    ['text', 'label']
]
test = df.loc[
    df['date'] >= '2024-01-01',
    ['text', 'label']
]
```

Category-Based Sampling

```
balanced = pd.concat([
    # Category-Based Sampling
    per_cat = 1000
    balanced = pd.concat([
        group.sample(n=min(per_cat,
            len(group))) # cap at what's
            available
        for _, group in
            df.groupby('category')
    ])
])
```

Quality Threshold Filtering

```
high_quality = df.loc[
    (df['rating'] > 4.0) &
    (df['confidence'] > 0.9),
    ['text', 'label', 'metadata']
]
```

Performance Considerations

Optimizing Filter Operations

When working with large datasets (millions of rows), filter performance matters. Pandas operations are generally fast, but some techniques are faster than others.

Fastest to Slowest:

1. Vectorized operations (boolean indexing)
2. `.query()` method
3. `.apply()` with lambda functions
4. Iterating with `.iterrows()`

For LLM datasets that might contain millions of training examples, always prefer vectorized operations and avoid row-by-row iteration.



Benchmark Tip: Use `%%timeit` in Jupyter notebooks to compare performance of different filtering approaches on your actual data.

Real-World Example: Filtering API Responses

Let's walk through a realistic scenario: you've scraped product reviews from an API and need to filter them for training a sentiment analysis model.

```
import pandas as pd

# Load API response data
reviews_df = pd.read_json('api_reviews.json')

# Step 1: Remove invalid entries
valid_reviews = reviews_df.query('rating >= 1 and rating <= 5 and text.notnull()')

# Step 2: Filter by language and length
english_reviews = valid_reviews.loc[
    (valid_reviews['language'] == 'en') &
    (valid_reviews['text'].str.len() > 20),
    ['text', 'rating', 'verified_purchase']
]

# Step 3: Balance the dataset — never request more than a bucket holds
def take(frame, n):
    return frame.sample(n=min(n, len(frame))) # cap at bucket size

positive = take(english_reviews.query('rating >= 4'), 5000)
neutral = take(english_reviews.query('rating == 3'), 5000)
negative = take(english_reviews.query('rating <= 2'), 5000)

training_data = pd.concat([positive, neutral, negative]).sample(frac=1.0)
```

Advanced Pattern: Dynamic Filtering

In production LLM pipelines, you often need to apply filters based on configuration files or user input. Here's a pattern for building dynamic filters:

```
def build_dynamic_filter(df, filter_config):
    """
    Apply filters based on a configuration dictionary.

    Args:
        df: Input DataFrame
        filter_config: Dict with filter criteria
    Returns:
        Filtered DataFrame
    """
    # Seed the mask ALIGNED TO df's index — so it still works when df has a
    # non-default index (after a prior filter, concat, or a custom id index).
    mask = pd.Series(True, index=df.index)

    if 'min_rating' in filter_config:
        mask &= df['rating'] >= filter_config['min_rating']

    if 'languages' in filter_config:
        mask &= df['language'].isin(filter_config['languages'])

    if 'date_range' in filter_config:
        start, end = filter_config['date_range']
        mask &= df['date'].between(start, end)

    if 'exclude_categories' in filter_config:
        mask &= ~df['category'].isin(filter_config['exclude_categories'])

    return df[mask]

# Usage
config = {
    'min_rating': 4.0,
    'languages': ['en', 'es'],
    'date_range': ('2024-01-01', '2024-12-31'),
}
filtered = build_dynamic_filter(df, config)
```

Debugging Filter Operations

1

Check Mask Size

```
mask = df['rating'] > 4
print(f"Rows selected: {mask.sum()}")
print(f"Percentage:
{mask.mean():.1%}")
```

2

Inspect Failed Conditions

```
# See which rows failed
failed = df[~mask]
print(failed['rating'].describe())
```

3

Validate Results

```
# Verify filter worked
result = df[mask]
assert result['rating'].min() > 4
```

📄 **Debug Tip:** Always check `mask.sum()` before applying it. If you get 0 rows, your conditions might be too strict or contain a logical error.

Working with Missing Data in Filters

Missing values (NaN) in your dataset can cause unexpected behavior with filters. By default, comparisons with NaN return False, which means rows with missing data are excluded.

```
# NaN values excluded by default
df[df['rating'] > 3.0] # Excludes NaN ratings

# Explicitly handle missing values
has_rating = df['rating'].notna()
high_rating = df['rating'] > 3.0
df[has_rating & high_rating]

# Or use fillna
df[df['rating'].fillna(0) > 3.0]
```

Common Patterns

```
# Include rows WITH missing values
missing_ratings = df['rating'].isna()

# Exclude rows with ANY missing value
complete_cases = df.dropna()

# Exclude rows missing specific columns
required = ['text', 'rating', 'label']
clean = df.dropna(subset=required)
```

For LLM training data, decide upfront whether to exclude, impute, or flag missing values—don't let them silently disappear.

Key Takeaways



Boolean Indexing is Fundamental

Master the pattern `df[condition]` and always use parentheses when combining conditions with `&`, `|`, or `~`



`.query()` for Readability

Use the `query` method when you have three or more conditions—it's cleaner and less error-prone than complex boolean expressions



Chain Masks for Clarity

Break complex filters into named intermediate masks to make your logic transparent and easier to debug



`.loc` for Precise Selection

Use `.loc` when you need to select both specific rows and columns, especially when preparing data for model training

Best Practices Checklist

✓ Do This

- Always use parentheses around each condition in boolean expressions
- Check `mask.sum()` to validate your filter before applying
- Use `.query()` for complex multi-condition filters
- Explicitly handle missing values with `.notna()` or `.fillna()`
- Document your filter logic with comments
- Use vectorized operations for performance

✗ Avoid This

- Forgetting parentheses in compound conditions
- Using `.iterrows()` for filtering operations
- Assuming missing values are automatically handled
- Chaining too many filters without intermediate validation
- Using `==` to check for NaN (use `.isna()` instead)
- Ignoring performance when working with large datasets



Thanks